Vinayak java

Q1.Given an integer, n, store its binary representation bin as a string. bin is an array of binary digits indexed from 0 to length(bin) - 1.

Reduce its binary representation to zero by using the following operations:

1. **Change the i^{th} binary digit only if the $(i+1)^{th}$ binary digit is 1 and all digits from $i+2$ to the end are zero.**
2. **Change the rightmost digit without restriction.**

What is the minimum number of operations required to complete this task?

```
// This function isn't provided in the prompt, but is needed to test the solution.
// It converts a long to its binary string representation.
public static String toBinaryString(long n) {
    if (n == 0) return "0";
    return Long.toBinaryString(n);
}

// This is the function signature from the HackerRank problem
public static long minOperations(long n) {
    String bin = toBinaryString(n);
    int len = bin.length();

    long t0 = 0; // Represents T_0(s) for the current suffix
    long t1 = 0; // Represents T_1(s) for the current suffix
    int k = 0; // Represents the length of the current suffix

    for (int i = len - 1; i >= 0; i--) {
        char c = bin.charAt(i);

        // We calculate the new t0 and t1 based on the
        // t0 and t1 of the suffix to the right (calculated in the prev loop)
        long new_t0 = 0;
        long new_t1 = 0;

        // Recurrence T_0("10...0" with k ones) = (1L << k) - 1
        long t0_helper_k_minus_1 = 0;
        if (k > 0) {
            if (k == 1) {
                t0_helper_k_minus_1 = 1; // T_0("1")
            } else {
                t0_helper_k_minus_1 = (1L << (k - 1)) - 1; // T_0("10...0"_{k-1})
            }
        }


        if (c == '0') {
            // T_0("0" + s) = T_0(s)
            new_t0 = t0;

            // T_1("0" + s) = T_1(s) + 1 + T_0("10...0"_{k-1})
            new_t1 = t1 + 1 + t0_helper_k_minus_1;

        } else { // c == '1'
            // T_0("1" + s) = T_1(s) + 2^k
            new_t0 = t1 + (1L << k);

            // T_1("1" + s) = T_0(s)
            new_t1 = t0;
        }

        t0 = new_t0;
        t1 = new_t1;
        k++;
    }

    return t0;
}
```

Q2. Given an array of n integers, arr, you can perform the following operation any number of times:

1. **Choose any index i ($0 \le i < n - 1$) and swap arr[i] and arr[i+1].**
2. **Each element can be swapped at most once during the process.**

The strength of an index i is defined as arr[i] * (i+1), using 0-based indexing. The total strength is the sum of strengths of all indices:

$$\text{Total Strength} = \sum_{i=0}^{n-1} arr[i] \times (i+1)$$
Find the maximum possible sum of the strength of all indices after optimal swaps.

```
public static long getMaximumSumOfStrengths(List<Integer> arr) {
    int n = arr.size();
    if (n == 0) return 0;
    if (n == 1) return (long) arr.get(0); // Only one element, no swaps possible

    // dp[i] = maximum total strength starting from index i
    long[] dp = new long[n + 2]; // +2 to safely handle dp[i+2] access
    Arrays.fill(dp, 0);

    // Process from right to left
    for (int i = n - 1; i >= 0; i--) {

        // Option 1: No swap at index i
        long noSwap = (long) arr.get(i) * (i + 1) + dp[i + 1];

        // Option 2: Swap arr[i] and arr[i+1], only if i+1 < n
        long swap = Long.MIN_VALUE;
        if (i + 1 < n) {
            swap = (long) arr.get(i + 1) * (i + 1) +
                   (long) arr.get(i) * (i + 2) +
                   dp[i + 2];
        }

        // Take the better of the two
        dp[i] = Math.max(noSwap, swap);
    }

    // dp[0] holds the maximum total strength for the whole array
    return dp[0];
}
```

Q3. Given an array arr of non-negative integers. In one operation, a positive integer x is chosen, and x is subtracted from all numbers in the array that are greater than or equal to x.
Find the number of different final arrays possible after applying the operation 0 or more times. Return the answer modulo $10^9 + 7$.

Note:

- **Different values of x can be chosen for different operations.**
- **Two final arrays are considered different if they differ in at least one position.**

```
public static int getCount(List<Integer> arr) {
    final long MOD = 1_000_000_007L;

    // Collect distinct positive elements
    TreeSet<Integer> distinct = new TreeSet<>();
    for (int num : arr) {
        if (num > 0) distinct.add(num);
    }

    // If all zeros → only one possible final array
    if (distinct.isEmpty()) return 1;

    long result = 1;
    int prev = 0;

    // Each gap (delta) adds (delta + 1) possible choices
    for (int val : distinct) {
        int delta = val - prev;
        result = (result * (delta + 1)) % MOD;
        prev = val;
    }

    return (int) result;
}
```

Q4. Implement a function that, for an array of integers arr and a subarray of size k, determines the maximum among the minimums of all contiguous subarrays of size k.

```
public static int maximumSubarrayMinimum(int[] arr, int k) {
    int n = arr.length;
    if (n == 0 || k == 0) {
        return 0; // Or handle as an error
    }

    Deque<Integer> dq = new ArrayDeque<>();
    int maxOfMins = Integer.MIN_VALUE;

    for (int i = 0; i < n; ++i) {
        // 1. Remove elements from the front that are out of the window
        if (!dq.isEmpty() && dq.peekFirst() <= i - k) {
            dq.pollFirst();
        }

        // 2. Remove elements from the back that are >= current element
        while (!dq.isEmpty() && arr[dq.peekLast()] >= arr[i]) {
            dq.pollLast();
        }

        // 3. Add current element's index to the back
        dq.offerLast(i);

        // 4. Once the window is full, record the minimum
        if (i >= k - 1) {
            maxOfMins = Math.max(maxOfMins, arr[dq.peekFirst()]);
        }
    }

    return maxOfMins;
}
```

Q5. You are building a backend service for a stock analytics platform. You have an array named stockPrices where stockPrices[i] denotes the price of the stock in the i^{th} month.
A bullish trend of size k is an interval of k months such that the stock prices increase strictly throughout those months.

Implement a function that calculates the number of bullish trends of size k.

```
public static int calculateBullishTrends(List<Integer> stockPrices, int k) {
    int n = stockPrices.size();
    if (k > n) {
        return 0;
    }

    // A trend of 1 is just the element itself
    if (k == 1) {
        return n;
    }

    int totalTrends = 0;
    int currentIncreasingLength = 1;

    for (int i = 1; i < n; ++i) {
        if (stockPrices.get(i) > stockPrices.get(i - 1)) {
            // Trend continues
            currentIncreasingLength++;
        } else {
            // Trend broken
            currentIncreasingLength = 1;
        }

        // If our current run is long enough, it counts as a trend
        if (currentIncreasingLength >= k) {
            totalTrends++;
        }
    }

    return totalTrends;
}
```

Q6. A cyber security firm has discovered a new type of encryption being used by a group of hackers. The encryption key will be a valid key, which is a number that has exactly 3 factors

(or divisors). For example, 4 is a valid key because it has exactly 3 factors: 1, 2, and 4. But 6 is not a valid key because it has 4 factors: 1, 2, 3, and 6.[2]
Given an array keys of length [3]n, find the num[4]ber of valid keys in the range $[1, \text{keys}[i]]$, both inclusive, for each $0 \le i < n$.

```
public class Solution {

    // We need to precompute prime squares
    private static final List<Long> primeSquares = new ArrayList<>();
    private static final int SIEVE_LIMIT = 5000001; // sqrt(2.5e13) is
~5,000,000
    private static boolean precomputed = false;

    private static void precomputePrimes() {
        if (precomputed) return; // Only run once

        boolean[] isPrime = new boolean[SIEVE_LIMIT];
        for (int i = 2; i < SIEVE_LIMIT; i++) {
            isPrime[i] = true;
        }

        for (long p = 2; p < SIEVE_LIMIT; ++p) {
            if (isPrime[(int)p]) {
                // p is prime, add its square
                primeSquares.add(p * p);

                // Mark multiples as not prime
                for (long i = p * p; i < SIEVE_LIMIT; i += p) {
                    isPrime[(int)i] = false;
                }
            }
        }
        precomputed = true;
    }

    // Helper function for binary search (finds upper bound)
    private static int upperBound(List<Long> arr, long target) {
        int low = 0;
        int high = arr.size();
        while (low < high) {
            int mid = low + (high - low) / 2;
            if (arr.get(mid) <= target) {
                low = mid + 1;
            } else {
                high = mid;
            }
        }
        return low;
    }

    public static List<Integer> getValidKey(long n, List<Long> keys) {
        // Ensure our precomputation is done
        precomputePrimes();

        List<Integer> result = new ArrayList<>();
        for (long r : keys) {
            // Find the count of prime squares <= r
            int count = upperBound(primeSquares, r);
            result.add(count);
        }
        return result;
    }
}
```

Q7. Design a system to extract a Table of Contents (TOC) from a document written in a simple markup language. The TOC should be structured based on the following rules:

1. **Chapter Titles:**
    - **A line that begins with a single # followed by a space (#) represents a chapter.**
2. **Section Titles:**
    - **A line that begins with a double ## followed by a space (##) represents a section within a chapter.**

The output should list chapter titles as top-level entries and section titles as indented sub-entries under their respective chapters.

```
    public static List<String>
tableOfContents(List<String> text) {

        List<String> toc = new ArrayList<>();

        int chapterNum = 0;

        int sectionNum = 0;



        for (String line : text) {

            // Check for "## "

            if (line.startsWith("## ")) {

                sectionNum++;

                String title = line.substring(3);

                toc.add("  " + chapterNum + "." + sectionNum
+ ". " + title);

            }

            // Check for "# "

            else if (line.startsWith("# ")) {

                chapterNum++;

                sectionNum = 0; // Reset section counter

                String title = line.substring(2);

                toc.add(chapterNum + ". " + title);
```

```
                }

                // Otherwise, ignore the line

            }

            return toc;

        }

    }
```

Q8. A disk stores hierarchical data in an undirected tree with tree_nodes nodes numbered from 0 to tree_nodes - 1, rooted at node 0. Each node has a character value associated with it, where arr[i] is the character on the i^{th} node.
You are given an array queries of size m. For each query queries[i], determine how many nodes w (including queries[i]) exist on the path from queries[i] to the root (node 0), such that the letters on the path from w to queries[i] can be arranged to form a palindrome.

Return an integer array of size m with the answer to each query.

```
public class Solution {


    static int[] parent;

    static List<List<Integer>> adj;


    // DFS to build the parent array

    private static void buildParents(int u, int p) {

        parent[u] = p;

        for (int v : adj.get(u)) {

            if (v != p) {

                buildParents(v, u);

            }

        }

    }


    public static List<Integer> palindromePaths(int tree_nodes, int
tree_edges,

                                                List<Integer> tree_from, List<Integer>
tree_to,

                                                String arr, List<Integer> queries) {


        // Build adjacency list

        adj = new ArrayList<>(tree_nodes);

        for (int i = 0; i < tree_nodes; i++) {

            adj.add(new ArrayList<>());

        }

        for (int i = 0; i < tree_edges; ++i) {

            adj.get(tree_from.get(i)).add(tree_to.get(i));

            adj.get(tree_to.get(i)).add(tree_from.get(i));

        }

        // Build parent map, rooting the tree at 0

        parent = new int[tree_nodes];

        buildParents(0, -1);

        List<Integer> result = new ArrayList<>();

        for (int v : queries) {

            int validCount = 0;

            int pathBitmask = 0;

            int currentNode = v;


            while (currentNode != -1) {

                // Get char and update bitmask

                char c = arr.charAt(currentNode);

                pathBitmask ^= (1 << (c - 'a'));


                // Check if mask can form a palindrome

                // (mask == 0) or (mask is a power of 2)

                if ((pathBitmask & (pathBitmask - 1)) == 0) {

                    validCount++;

                }
```

```
            // Break if we just processed the root
            if (currentNode == 0) break;

            // Move to parent
            currentNode = parent[currentNode];
        }
        result.add(validCount);
    }
    return result;
  }
}
```

—----------

```
import java.util.*;

public class Solution {

  static int[] parent;
  static List<List<Integer>> adj;

  // DFS to populate parent array
  private static void buildParents(int node, int par) {
    parent[node] = par;
    for (int neighbor : adj.get(node)) {
      if (neighbor != par) {
        buildParents(neighbor, node);
      }
    }
  }

  public static List<Integer> palindromePaths(
    int tree_nodes, int tree_edges,
    List<Integer> tree_from, List<Integer> tree_to,
    String arr, List<Integer> queries) {

    // Step 1: Build adjacency list
    adj = new ArrayList<>(tree_nodes);
    for (int i = 0; i < tree_nodes; i++) {
      adj.add(new ArrayList<>());
    }
    for (int i = 0; i < tree_edges; i++) {
      int u = tree_from.get(i);
      int v = tree_to.get(i);
      adj.get(u).add(v);
      adj.get(v).add(u);
    }

    // Step 2: Build parent array with DFS (root = 0)
    parent = new int[tree_nodes];
    buildParents(0, -1);

    // Step 3: Process each query
    List<Integer> result = new ArrayList<>();
    for (int queryNode : queries) {
      int count = 0;
      int mask = 0;
      int curr = queryNode;

      // Move upward to root
      while (curr != -1) {
        // Flip bit corresponding to this node's character
        char c = arr.charAt(curr);
        mask ^= (1 << (c - 'a'));
```

```
            // Check palindrome condition
            if ((mask & (mask - 1)) == 0) {
                count++;
            }

            // Move to parent
            curr = parent[curr];
        }

        result.add(count);
    }

    return result;
}

// Example usage
public static void main(String[] args) {
    int n = 4;
    String arr = "abaa";
    List<Integer> from = List.of(0, 1, 0);
    List<Integer> to = List.of(1, 3, 2);
    List<Integer> queries = List.of(1, 2);

    List<Integer> result = palindromePaths(n, from.size(), from, to,
arr, queries);
    System.out.println(result); // Output: [1, 2]
}
```

Q11. A wallet contains money in various denominations. You can modify any denomination by spending $1 to:

- **Convert it to any value in the range `[money[i]+1, 2 * money[i]]`**
- **Convert it to any value in the range `[ceiling(money[i]/2), money[i]-1]`**

Calculate the minimum amount needed to make all denominations equal.

```
}
```

Q10. Valid BST Permutations

```
#include <vector>

/*
 * Complete the 'numBST' function below.
 *
 * The function is expected to return a LONG.
 * The function accepts INTEGER n as parameter.
 */

long numBST(int n) {
    // This problem is solved by the n-th Catalan Number.
    // We can use dynamic programming to calculate it.

    // dp[i] will store the number of unique BSTs with 'i' nodes.
    // We use long to avoid integer overflow for larger n, as requested.
    std::vector<long> dp(n + 1, 0);

    // Base cases
    // There is 1 way to make an empty tree (0 nodes)
    dp[0] = 1;
    // There is 1 way to make a tree with 1 node
    dp[1] = 1;

    // Fill the dp table from 2 up to n
    for (int i = 2; i <= n; ++i) {
        // To calculate dp[i] (BSTs with i nodes), we iterate through
        // all possible root nodes (from 1 to i).

        // Let's say we pick 'j' as the root (where j goes from 1 to i).
        // The nodes in the left subtree will be (1...j-1), which is 'j-1'
nodes.
        // The nodes in the right subtree will be (j+1...i), which is 'i-j'
nodes.
        for (int j = 1; j <= i; ++j) {
            int leftNodes = j - 1;
            int rightNodes = i - j;

            // The total combinations for this root 'j' is:
            // (ways to form left subtree) * (ways to form right subtree)
            dp[i] += dp[leftNodes] * dp[rightNodes];
        }
    }

    // The final answer is the number of BSTs with n nodes
    return dp[n];
}
```

```
import java.util.List;

class Solution {

    /**
     * Calculates the minimum cost to change 'from' to 'to'.
     */
    private int getCost(int from, int to) {
        if (from == to) {
            return 0;
        }

        int cost = 0;
        if (from < to) {
            // Cost to go up
            while (from < to) {
                from *= 2;
                cost++;
            }
        } else {
            // Cost to go down
            while (from > to) {
                // Equivalent to ceiling(from / 2.0)
                from = (from + 1) / 2;
                cost++;
            }
        }
        return cost;
    }

    /**
     * Calculates the minimum cost to make all denominations equal.
     * Function name from the image: 'minCost'
     */
    public long minCost(List<Integer> money) {
        long minTotalCost = Long.MAX_VALUE;

        // The optimal target is likely near the values present.
        // The C++ in the image iterates 1..29, suggesting a small, fixed range.
        // We will iterate from 1 to a reasonable upper bound (e.g., 100).
        // A more robust solution would determine this range from input constraints.
        int maxTargetToTry = 100;

        for (int target = 1; target <= maxTargetToTry; target++) {
            long currentTotalCost = 0;
            for (int val : money) {
                currentTotalCost += getCost(val, target);
            }
            minTotalCost = Math.min(minTotalCost, currentTotalCost);
        }

        return minTotalCost;
    }
}
```

Q12. Developers are optimizing their horizontal pod autoscaler for their microservices. There are n microservices, and the number of pods for the P^{th} microservice is pods[i].

There is an event log of size m, described as a 2D array logs where logs[j] is an array of integers of size 3. The logs have the following interpretations:

- **[1, p, x]: The number of pods for the p^{th} microservice is changed to x (p is a 0-based index).**
- **[2, l, x]: All microservices whose number of pods is less than x are changed to x. (Based on the description, the second value l is ignored).**

Your task is to find the resulting number of pods for each microservice after processing all the logs.

```
import java.util.List;
import java.util.ArrayList;

class Solution {

    /**
     * Processes logs to find the final pod counts.
     * Function name from the image: 'findPodCount'
     */
    public List<Integer> findPodCount(List<Integer> pods, List<List<Integer>> logs) {

        // Create a mutable copy of the pods list
        List<Integer> currentPods = new ArrayList<>(pods);

        for (List<Integer> log : logs) {
            int type = log.get(0);

            if (type == 1) {
                // Type 1: Set specific microservice
                int p_index = log.get(1);
                int x_value = log.get(2);

                if (p_index >= 0 && p_index < currentPods.size()) {
                    currentPods.set(p_index, x_value);
                }

            } else if (type == 2) {
                // Type 2: Set all pods less than x to x
                // We assume log.get(1) is ignored and log.get(2) is 'x'
                int x_value = log.get(2);

                for (int i = 0; i < currentPods.size(); i++) {
                    if (currentPods.get(i) < x_value) {
                        currentPods.set(i, x_value);
                    }
                }
            }
        }

        return currentPods;
    }
}
```

Q13. Number of Connected Components in an Undirected Graph

You have a graph of `n` nodes. You are given an integer `n` and an array `edges` where `edges[i] = [a, b]` indicates that there is an edge between `a` and `b` in the graph.

Return the number of connected components in the graph.

```
class Solution {

    // Inner class for the Union-Find data structure
    private class UnionFind {
        private int[] parent;
        private int[] rank;
        private int count;

        public UnionFind(int n) {
            this.count = n;
            this.parent = new int[n];
            this.rank = new int[n];
            for (int i = 0; i < n; i++) {
                parent[i] = i; // Each node is its own parent initially
                rank[i] = 1;   // Initial rank is 1
            }
        }

        /**
         * Finds the representative (root) of the set containing element i,
         * with path compression.
         */
        public int find(int i) {
            if (parent[i] == i) {
                return i;
            }
            // Path compression
            parent[i] = find(parent[i]);
            return parent[i];
        }

        /**
         * Unites the sets containing elements i and j using union by rank.
         * Returns true if a merge occurred (components were different),
         * false otherwise.
         */
        public boolean union(int i, int j) {
            int rootI = find(i);
            int rootJ = find(j);

            if (rootI != rootJ) {
                // Union by rank
                if (rank[rootI] > rank[rootJ]) {
                    parent[rootJ] = rootI;
                } else if (rank[rootI] < rank[rootJ]) {
                    parent[rootI] = rootJ;
                } else {
                    parent[rootJ] = rootI;
                    rank[rootI]++;
                }
                count--; // Decrement component count
                return true;
            }
            return false; // Already in the same component
        }

        public int getCount() {
            return count;
        }
    }

    /**
     * Function from the image: 'countComponents'
     */
    public int countComponents(int n, int[][] edges) {
        if (n <= 0) {
            return 0;
        }

        UnionFind uf = new UnionFind(n);

        for (int[] edge : edges) {
            uf.union(edge[0], edge[1]);
        }

        return uf.getCount();
    }
}
```

CPP solutioins

Shortest route

Q1. There are roads_nodes buildings and m roads in HackerLand.
The i-th road connects building roads_from[i] and roads_to[i], is bidirectional, and has length roads_weight[i].

A delivery person has to deliver an order before returning back to the office.
The delivery person starts at building a, the order has to be delivered at building c, and the office is at building b.
If a road is traveled multiple times, its length will be counted only once.

You need to find the shortest route from a to b via c.
If there is no such path, return -1.

```
#include <iostream>
#include <vector>
#include <queue>
#include <limits>

using namespace std;

using Path = pair<long long, int>;

const long long INFINITY_PATH = numeric_limits<long long>::max();

vector<long long> runDijkstra(int startNode, int numNodes, const vector<vector<pair<int, int>>>& adjacencyList) {
    vector<long long> distances(numNodes, INFINITY_PATH);
    distances[startNode] = 0;

    priority_queue<Path, vector<Path>, greater<Path>> priorityQueue;
    priorityQueue.push({0, startNode});

    while (!priorityQueue.empty()) {
        long long currentDistance = priorityQueue.top().first;
        int currentNode = priorityQueue.top().second;
        priorityQueue.pop();

        if (currentDistance > distances[currentNode]) {
            continue;
        }

        for (const auto& edge : adjacencyList[currentNode]) {
            int neighborNode = edge.first;
            int edgeWeight = edge.second;
            long long newDistance = currentDistance + edgeWeight;

            if (newDistance < distances[neighborNode]) {
                distances[neighborNode] = newDistance;
                priorityQueue.push({newDistance, neighborNode});
            }
        }
    }
    return distances;
}

long getShortestRoute(int roads_nodes, vector<int> roads_from, vector<int> roads_to, vector<int> roads_weight, int a, int b, int c) {

    int numBuildings = roads_nodes;
    int numRoads = roads_from.size();

    vector<vector<pair<int, int>>> adjacencyList(numBuildings);
    for (int i = 0; i < numRoads; ++i) {
        int nodeU = roads_from[i];
        int nodeV = roads_to[i];
        int weight = roads_weight[i];

        adjacencyList[nodeU].push_back({nodeV, weight});
        adjacencyList[nodeV].push_back({nodeU, weight});
    }

    vector<long long> distancesFromA = runDijkstra(a, numBuildings, adjacencyList);
    long long path_A_to_B = distancesFromA[b];

    vector<long long> distancesFromB = runDijkstra(b, numBuildings, adjacencyList);
    long long path_B_to_C = distancesFromB[c];

    if (path_A_to_B == INFINITY_PATH || path_B_to_C == INFINITY_PATH) {
        return -1;
    }

    return path_A_to_B + path_B_to_C;
}
```

Q2. 1. Binary Manipulation

This solution finds the minimum operations by iterating through the binary string. The logic determines that if the current bit doesn't match the target state (initially '0'), it adds the operations needed to flip it (2k, where k is remaining bits) and sets the target for the rest of the string to '1' (to form a

`10...0` pattern). If it matches, the target for the rest of the string becomes '0'.

```
#include <iostream>
#include <string>
#include <bitset>
#include <algorithm>
#include <vector>

using namespace std;

long minOperations(long n) {
    if (n == 0) {
        return 0;
    }

    string binaryString = bitset<64>(n).to_string();

    size_t firstOne = binaryString.find('1');
    if (firstOne != string::npos) {
        binaryString = binaryString.substr(firstOne);
    }

    long long totalOperations = 0;
    bool targetIsOne = false;
    int numBits = binaryString.length();

    for (int i = 0; i < numBits; ++i) {
        int remainingBits = numBits - 1 - i;
        char currentBit = binaryString[i];

        bool isMismatch = (currentBit == '1' && !targetIsOne) ||
                          (currentBit == '0' && targetIsOne);

        if (isMismatch) {
            totalOperations += (1LL << remainingBits);
            targetIsOne = true;
        } else {
            targetIsOne = false;
        }
    }

    return totalOperations;
}
```

Q3. **You are given n points in a 2D plane. For each point i (from 1 to n), you are given:**
• Its x-coordinate, x[i]
• Its y-coordinate, y[i]
• An associated weight, premium[i]

Your task is to find a single "optimal" point (cx, cy) in the 2D plane that minimizes the total weighted Manhattan distance to all n given points.
The total weighted Manhattan distance, S, is defined by the following summation:
You need to find the coordinates cx and cy that result in the smallest possible value for S

```
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

ll weightedMedian(vector<pair<int,int> >& a) {
    sort(a.begin(), a.end());
    ll total = 0;
    for (int i = 0; i < (int)a.size(); ++i)
        total += a[i].second;

    ll half = (total + 1) / 2, prefix = 0;
    for (int i = 0; i < (int)a.size(); ++i) {
        prefix += a[i].second;
        if (prefix >= half)
            return a[i].first;
    }
    return a.back().first; // fallback
}

ll totalWeightedManhattan(vector<int>& x, vector<int>& y,
vector<int>& w, ll cx, ll cy) {
    ll sum = 0;
    for (int i = 0; i < (int)x.size(); ++i)
        sum += 1LL * w[i] * (abs(x[i] - cx) + abs(y[i] - cy));
    return sum;
}
```

Q3. You are given an array arr containing n positive integers.
Your goal is to make all elements in the array equal to some single target value T.
To do this, you can perform two types of operations on any element x in the array. Each operation costs 1 step:
1. Increase Operation: Replace x with any integer y such that x+1 <= y <= 2x.
2. Decrease Operation: Replace x with any integer y such that ceil(x/2) <= y <= x-1.
You need to find a single target value T (which you are free to choose) and then transform every element in arr to T using the minimum possible number of steps for each transformation.
Your final answer is the minimum total steps required.
Formally, you must find a target T that minimizes the following cost function:

where steps(a, b) is the minimum number of operations required to transform the starting value a into the target value b.
Constraints
• 1 <= n <=10^5 (Number of elements in the array)
• 1 <= arr[i] <= 10^6 (Value of each element)

Is question ka test cases and output ni mila merko
Not sure answer will work ot not

```
#include <bits/stdc++.h>
using namespace std;

const int MAXV = 1000000;
const int MAX_DEPTH = 20;   // Limit BFS depth (enough for convergence)

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<int> arr(n);
    for (int i = 0; i < n; ++i) cin >> arr[i];

    vector<int> visit_version(MAXV + 1, -1);
    vector<int> visit_count(MAXV + 1, 0);
    vector<long long> total_dist(MAXV + 1, 0);

    int version = 0;

    for (int x : arr) {
        ++version;
        queue<pair<int,int>> q;
        q.push({x, 0});
        visit_version[x] = version;

        while (!q.empty()) {
            int cur = q.front().first;
            int step = q.front().second;
            q.pop();

            visit_count[cur]++;
            total_dist[cur] += step;

            if (step >= MAX_DEPTH) continue;

            // decrease operation: [ceil(cur/2), cur-1]
            int low = (cur + 1) / 2;
            for (int next = low; next < cur; ++next) {
                if (next < 1) break;
                if (visit_version[next] == version) continue;
                visit_version[next] = version;
                q.push({next, step + 1});
            }

            // increase operation: [cur+1, 2*cur]
            int high = min(MAXV, cur * 2);
            for (int next = cur + 1; next <= high; ++next) {
                if (visit_version[next] == version) continue;
                visit_version[next] = version;
                q.push({next, step + 1});
            }
        }
    }

    long long best = LLONG_MAX;
    for (int i = 1; i <= MAXV; ++i) {
        if (visit_count[i] == n)
            best = min(best, total_dist[i]);
    }

    cout << best << "\n";
    return 0;
}
```

Q4. **You are given:**
1. A string s of length n, which consists only of digit characters ('0' through '9').
2. An integer k.
Your task is to find and return the total number of substrings of s that satisfy a specific condition.
The Condition
A substring is considered "valid" if every single character that appears in that substring has a frequency exactly equal to k.
If a character is not present in the substring (i.e., its frequency is 0), it is ignored.

```
#include <iostream>
#include <string>
#include <vector>

using namespace std;

void addChar(int c, int k, vector<int>& freq, int&
distinctCount, int& countAtK) {
    if (freq[c] == 0) {
        distinctCount++;
    }
    freq[c]++;
    if (freq[c] == k) {
        countAtK++;
    }
    else if (freq[c] == k + 1) {
        countAtK--;
    }
}
```

```
void removeChar(int c, int k, vector<int>& freq, int&
distinctCount, int& countAtK) {
    if (freq[c] == k + 1) {
        countAtK++;
    }
    else if (freq[c] == k) {
        countAtK--;
    }
    freq[c]--;
    if (freq[c] == 0) {
        distinctCount--;
    }
}

long long countValidSubstrings(string s, int k) {
    int n = s.length();
    long long totalValidSubstrings = 0;

    for (int m = 1; m <= 10; ++m) {

        int windowLen = m * k;
        if (windowLen > n) {
            break;
        }

        vector<int> freq(10, 0);
        int distinctCount = 0;
        int countAtK = 0;

        for (int j = 0; j < windowLen; ++j) {
            addChar(s[j] - '0', k, freq, distinctCount, countAtK);
        }

        if (distinctCount == m && countAtK == m) {
            totalValidSubstrings++;
        }

        for (int i = 1; i <= n - windowLen; ++i) {
            int j = i + windowLen - 1;

            removeChar(s[i - 1] - '0', k, freq, distinctCount,
countAtK);

            addChar(s[j] - '0', k, freq, distinctCount, countAtK);

            if (distinctCount == m && countAtK == m) {
                totalValidSubstrings++;
            }
        }
    }

    return totalValidSubstrings;
}
```

Q5**. ou are given two undirected and unweighted graphs, G1 and G2.**
• G1 has n_1 nodes (V1) and a set of edges (E1).
• G2 has n_2 nodes (V2) and a set of edges (E2).
The distance between any two nodes is the number of edges in the shortest path between them.
Task
You must connect the two graphs by adding one new "bridge" edge (v_1, v_2), where v_1 is any node from G1 and v_2 is any node from G2, such that path from any node to any node is minimum Output Return the maximum path distance and the minimum path distance from the resultant graph

Not test cases available so based on AI

```
#include <iostream>
#include <vector>
#include <queue>
#include <algorithm>
#include <limits>

using namespace std;

const int INF = numeric_limits<int>::max();

/**
 * @brief Runs a BFS to find the farthest node from a given
start node.
 * @return A pair: {farthest_node, distance_to_farthest_node}
 */
pair<int, int> run_bfs(int startNode, int n, const
vector<vector<int>>& adj) {
    if (startNode == 0) return {0, 0};

    vector<int> distance(n + 1, -1);
    queue<int> q;

    q.push(startNode);
    distance[startNode] = 0;

    int farthest_node = startNode;
    int max_dist = 0;

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        if (distance[u] > max_dist) {
            max_dist = distance[u];
            farthest_node = u;
        }

        for (int v : adj[u]) {
```

```
            if (distance[v] == -1) {
                distance[v] = distance[u] + 1;
                q.push(v);
            }
        }
    }
    return {farthest_node, max_dist};
}

/**
 * @brief Calculates the Radius and Diameter of a tree in
O(N+M) time.
 * @return A pair: {Radius, Diameter}. Returns {INF, INF} if
graph is empty.
 */
pair<int, int> get_tree_metrics(int n, const
vector<vector<int>>& adj) {
    if (n == 0) return {0, 0};
    if (n == 1) return {0, 0};

    int first_node = 0;
    for(int i = 1; i <= n; ++i) {
        if(!adj[i].empty()) {
            first_node = i;
            break;
        }
    }
    if (first_node == 0) return {0, 0}; // Graph has nodes but no
edges

    // 1. Run BFS from an arbitrary node to find one end of the
diameter
    pair<int, int> first_bfs = run_bfs(first_node, n, adj);
    int node_A = first_bfs.first;

    // 2. Run BFS from that end-node to find the other end
    pair<int, int> second_bfs = run_bfs(node_A, n, adj);

    // 3. The distance is the diameter
    int diameter = second_bfs.second;

    // 4. The radius is ceil(diameter / 2)
    int radius = (diameter + 1) / 2;

    return {radius, diameter};
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    // --- Read Graph 1 ---
    int n1, m1;
    cin >> n1 >> m1;
    vector<vector<int>> adj1(n1 + 1);
    for (int i = 0; i < m1; ++i) {
        int u, v;
        cin >> u >> v;
        adj1[u].push_back(v);
        adj1[v].push_back(u);
    }

    // --- Read Graph 2 ---
    int n2, m2;
    cin >> n2 >> m2;
    vector<vector<int>> adj2(n2 + 1);
    for (int i = 0; i < m2; ++i) {
        int u, v;
        cin >> u >> v;
        adj2[u].push_back(v);
        adj2[v].push_back(u);
    }

    // --- Calculate Metrics ---
    pair<int, int> metrics1 = get_tree_metrics(n1, adj1);
    pair<int, int> metrics2 = get_tree_metrics(n2, adj2);

    int radius1 = metrics1.first;
    int diameter1 = metrics1.second;
    int radius2 = metrics2.first;
    int diameter2 = metrics2.second;

    // --- Calculate Final Answer ---

    // The minimized "maximum path distance" (new diameter)
    int new_diameter = max({diameter1, diameter2, radius1 +
radius2 + 1});

    // The "minimum path distance" is 1 (the bridge edge)
    int new_min_path = 1;

    cout << new_diameter << "\n";
    cout << new_min_path << "\n";

    return 0;
}
```

Harsh

Find Minimum Diameter After Merging Two Trees
Python:

```
from collections import deque
from math import ceil

class Solution:
    def minimumDiameterAfterMerge(self, e1, e2):
        n1 = len(e1) + 1
        n2 = len(e2) + 1

        g1 = self.build(n1, e1)
        g2 = self.build(n2, e2)

        d1 = self.diam(n1, g1)
        d2 = self.diam(n2, g2)

        cd = ceil(d1 / 2) + ceil(d2 / 2) + 1
        return max(d1, d2, cd)

    def build(self, n, edges):
        g = [[] for _ in range(n)]
        for u, v in edges:
            g[u].append(v)
            g[v].append(u)
        return g

    def diam(self, n, g):
        q = deque()
        deg = [0] * n

        for i in range(n):
            deg[i] = len(g[i])
            if deg[i] == 1:
                q.append(i)

        rem = n
        layers = 0

        while rem > 2:
            sz = len(q)
            rem -= sz
            layers += 1

            for _ in range(sz):
                u = q.popleft()
                for v in g[u]:
                    deg[v] -= 1
                    if deg[v] == 1:
                        q.append(v)
        return 2 * layers + (1 if rem == 2 else 0)
```

Disk Storage – Palindromic Path Prefix Queries
C++:

```
#include <bits/stdc++.h>
using namespace std;

void dfs(int id, vector<vector<int>>&adj,
unordered_map<int,int>&m2c, vector<int>&i2ans,
vector<int>&i2m, string &arr, int par){
  int ma = (par==-1) ? 0 : i2m[par];
  ma = ma ^ (1<<(arr[id]-'a'));

  int ans = 0;
  if (m2c.count(ma)) ans+=m2c[ma];

  for (int i=0; i<26; i++){
     int mv = (1<<i)^ma;
     if (m2c.count(mv)){
       ans+=m2c[mv];
     }
  }

  i2ans[id] = ans;
  m2c[ma]++;
  i2m[id] = ma;

  for (int &v: adj[id]){
    dfs(v,adj,m2c, i2ans, i2m, arr, id);
  }

  m2c[ma]--;
  if (m2c[ma]==0) m2c.erase(ma);
}

void solve(int n, int m, vector<int>& tree_from,
vector<int>& tree_to, string& arr, vector<int>& queries){
    vector<vector<int>>adj(n);

    for (int i=0; i<m; i++){
      adj[tree_from[i]].push_back(tree_to[i]);
    }

    unordered_map<int,int>m2c;
    m2c[0]++;
    vector<int>i2m(n,0);
    vector<int>i2ans(n,0);
```

```
    dfs(0, adj, m2c, i2ans, i2m, arr, -1);

    for (int &q: queries){
      cout<<i2ans[q]<<" ";
    }
    cout<<endl;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, tree_edges;
    cin >> n >> tree_edges;

    vector<int> tree_from(tree_edges);
    vector<int> tree_to(tree_edges);

    for (int i = 0; i < tree_edges; i++)
        cin >> tree_from[i];
    for (int i = 0; i < tree_edges; i++)
        cin >> tree_to[i];

    string arr;
    cin>>arr;

    int m;
    cin >> m;
    vector<int> queries(m);
    for (int i = 0; i < m; i++) cin >> queries[i];

    solve(n, tree_edges, tree_from, tree_to, arr, queries);

    return 0;
}
```

Number of Connected Components in an Undirected Graph

Pythton:

```
class Solution:
    def countComponents(self, n: int, edges: List[List[int]])
-> int:
        if not edges:
            return n==1

        adj_list = {i:[] for i in range(n)}
        visited=set()

        def dfs(node):
            visited.add(node)
            for neighbor in adj_list[node]:
                if neighbor not in visited:
                    dfs(neighbor)


        for x,y in edges:
            adj_list[x].append(y)
            adj_list[y].append(x)

        ans=0

        for i in range(n):
            if i not in visited:
                dfs(i)
                ans+=1

        return ans
```

Java:

```
public class Solution {

    private int find(int[] representative, int vertex) {
        if (vertex == representative[vertex]) {
            return vertex;
        }

        return representative[vertex] = find(representative,
representative[vertex]);
    }

    private int combine(int[] representative, int[] size, int
vertex1, int vertex2) {
        vertex1 = find(representative, vertex1);
        vertex2 = find(representative, vertex2);

        if (vertex1 == vertex2) {
            return 0;
        } else {
            if (size[vertex1] > size[vertex2]) {
                size[vertex1] += size[vertex2];
                representative[vertex2] = vertex1;
            } else {
                size[vertex2] += size[vertex1];
                representative[vertex1] = vertex2;
            }
            return 1;
        }
    }

    public int countComponents(int n, int[][] edges) {
        int[] representative = new int[n];
        int[] size = new int[n];
```

```
        for (int i = 0; i < n; i++) {
            representative[i] = i;
            size[i] = 1;
        }

        int components = n;
        for (int i = 0; i < edges.length; i++) {
            components -= combine(representative, size,
edges[i][0], edges[i][1]);
        }

        return components;
    }
}
```

C++:

```
class Solution {
public:
    int find(vector<int> &representative, int vertex) {
        if (vertex == representative[vertex]) {
            return vertex;
        }

        return representative[vertex] = find(representative,
representative[vertex]);
    }

    int combine(vector<int> &representative, vector<int>
&size, int vertex1, int vertex2) {
        vertex1 = find(representative, vertex1);
        vertex2 = find(representative, vertex2);

        if (vertex1 == vertex2) {
            return 0;
        } else {

            if (size[vertex1] > size[vertex2]) {
                size[vertex1] += size[vertex2];
                representative[vertex2] = vertex1;
            } else {
                size[vertex2] += size[vertex1];
                representative[vertex1] = vertex2;
            }
            return 1;
        }
    }

    int countComponents(int n, vector<vector<int>>&
edges) {
        vector<int> representative(n), size(n);

        for (int i = 0; i < n; i++) {
            representative[i] = i;
            size[i] = 1;
        }

        int components = n;
        for (int i = 0; i < edges.size(); i++) {
            components -= combine(representative, size,
edges[i][0], edges[i][1]);
        }

        return components;
    }
};
```